

CS247 – SPRING 26 – TUTORIAL 1

M2 – ADT DESIGN

ADT Motivation - Example

```
// Without ADT: a typedef looks safe, gives nothing
typedef int DayOfWeek;
DayOfWeek d = 42;           // Compiles. Completely wrong. What day of the week is 42?
d = d * 1000;               // Compiles. Nonsensical to multiply days of the week.
```

```
// With ADT: a proper class
#include <stdexcept>
class DayOfWeek {
    int day_;                // 0 = Sunday ... 6 = Saturday
public:
    explicit DayOfWeek(int d) {
        if (d < 0 || d > 6) throw std::out_of_range("Invalid day");
        day_ = d;
    }
    int get() const { return day_; }
};

DayOfWeek d(42);             // Throws. Caught immediately.
DayOfWeek d(3);             // 3 is Wednesday. Valid.
```

ADT Motivation - Challenge

What protection does `typedef` give you?

```
typedef int StudentID;
StudentID id1 = 12345;
StudentID id2 = -9999;           // Is a negative ID caught?
StudentID result = id1 + id2;    // Does adding IDs even make sense?

void enroll(StudentID id, int courseCode) { /* ... */ }
enroll(courseCode, id1);         // Arguments swapped: is this caught?
```

Answer: None at all.

Negative IDs, nonsensical arithmetic, and swapped function arguments all compile and run silently.

A proper `StudentID` ADT class would turn all three of these into compile-time errors.

This is the entire motivation for ADTs.

Member Initialization List (MIL) - Challenge

What is the value of `result_` after Tricky `t(5)`?

```
class Tricky {  
    int result_;  
    int base_;  
public:  
    Tricky(int b) : base_{b}, result_{base_ * 2} {}  
};  
  
Tricky t(5);
```

Answer: `result_` is NOT 10.

Members initialise in **declaration order**, so `result_` is initialised first, using `base_`, which hasn't been set yet (garbage value). **This is undefined behaviour.**

Fix: declare `base_` before `result_` in the class body.

Constant Member Functions - Challenge

Can a `const` method modify a member variable?

```
class Rational {  
    int num_, denom_;  
    mutable int accessCount_ = 0;  
public:  
    int getnum() const {  
        accessCount_++; // modifying inside a const method?  
        return num_;  
    }  
};
```

Answer: Yes, `mutable` exempts a member from the `const` restriction.

A `mutable` member can be modified inside a `const` method.

This is appropriate for caches and counters that don't affect the logical state. **Use moderately!**

Function Overloading - Challenge

Can two functions differ only in return type?

```
class Rational {  
public:  
    int  getValue(); // returns int  
    double getValue(); // returns double  
};  
  
Rational r(3, 4);  
auto x = r.getValue(); // Which version is called?
```

Answer: It does NOT compile.

Overloading on return type alone is illegal => the compiler uses argument types to choose, not the return type.

Since both `getValue()` take no arguments, the compiler cannot distinguish them and **reports a redefinition error**.

Default Arguments - Challenge

How do default arguments interact with implicit conversion?

```
class Rational {  
public:  
    Rational(int num, int denom = 1) { // NOT explicit  
        if (denom == 0)  
            throw std::out_of_range("...");  
    }  
};  
  
void print(Rational r) { /* ... */ }  
  
print(5); // Does this compile? What happens?
```

Answer: Yes, it compiles. `denom` has a default, so `Rational(int, int=1)` can be called with one argument.

Without `explicit`, the compiler silently converts `5` to `Rational(5, 1)` => `print(5)` is not exactly the same as `print(Rational(5))`, be careful.

If unintended, it's a silent bug. Else, mark the constructor `explicit`.

Default Arguments – Challenge 2

What is the difference between `()` and `{}` when calling an explicit constructor?

```
Rational m(2); // m = 2/1 (valid)
```

```
Rational m(2.3); // What happens?
```

```
Rational m{2.3}; // What about this?
```

Answer: `m(2.3)` compiles silently $\Rightarrow$ `2.3` narrows to `int 2` (truncated), giving `m = 2/1`. At most, you might get a warning (like `-Wconversion` or `-Wnarrowing`), but it will still compile.

`m{2.3}` is a compile error $\Rightarrow$ brace initialisation prohibits narrowing conversions.

Prefer `{}` to catch truncation bugs at compile time.

Operator Overloading - Example

```
// Nonmember operator+: handles r+s, 5+r, and r+5

Rational operator+(const Rational& lhs, const Rational& rhs) {
    return Rational{
        lhs.getnum() * rhs.getdenom() + lhs.getdenom() * rhs.getnum(),
        lhs.getdenom() * rhs.getdenom()
    };
}

Rational operator+(int lhs, const Rational& rhs) {
    return Rational(lhs) + rhs;
}

Rational operator+(const Rational& lhs, int rhs) {
    return lhs + Rational(rhs);
}
```

Operator Overloading - Challenge

The Post-Increment Lie

```
// Student wants m++ to add 1 to a Rational
const Rational& operator++(Rational& r, int) {
    r.setnum(r.getnum() + r.getdenom()); // adds denom/denom = 1
    return r;
}

Rational m(2, 1);           // m = 2/1

Rational old = m++;         // save the "old" value before incrementing

cout << old << endl;        // prints: ???
cout << m << endl;          // prints: ???
```

Answer: No, `old` and `m` both print `3/1`. The "old" value was never actually saved.

Post-increment has a specific contract: **save the old value** → **increment** → **return the old value**.

This implementation breaks the contract in two ways:

- It increments `r` first, then returns a reference to the already-modified `r`. There is no `old` value anywhere, it was overwritten before anything was saved.
- Returning `const Rational&` (a reference) means `old` binds to `r` itself, not a copy. So, `old` and `m` are effectively the same object.

Operator Overloading – Challenge (continue)

The correct implementation returns by value (a copy taken *before* the increment):

```
Rational operator++(Rational& r, int) {           // returns by VALUE, not reference
    Rational old = r;                             // snapshot FIRST
    r.setnum(r.getnum() + r.getdenom());          // then increment
    return old;                                    // return the snapshot
}

Rational m(2, 1);
Rational old = m++;

cout << old << endl;                             // 2/1 ✓ the value before increment
cout << m << endl;                                // 3/1 ✓ the incremented value
```

Pre-increment (`++m`) returns `Rational&`, a reference to the modified object.

Post-increment (`m++`) returns `Rational`, a copy of the object as it was before the change.

Mixing these up compiles silently and produces the wrong semantics.

Streaming Operators (<< and >>) - Example

```
std::ostream& operator<<(std::ostream& out, const Rational& r) {
    out << r.getnum() << '/' << r.getdenom();
    return out; // must return stream for chaining
}
std::istream& operator>>(std::istream& in, Rational& r) {
    int num, denom; char slash;
    in >> num >> slash >> denom; // reads "2/7"
    r = Rational(num, denom);
    return in;
}
// Chaining: each << returns cout for the next <<
cout << r << " + " << s << " = " << r + s << endl;
```

Streaming Operators (<< and >>) - Challenge

What does returning void from `operator<<` break?

```
void operator<<(std::ostream& out, const Rational& r) {  
    out << r.getnum() << '/' << r.getdenom();  
    // forgot to return anything, used void  
}  
  
Rational r(2, 7), s(1, 3);  
cout << r << " + " << s << endl; // What happens?
```

Answer: Compile error. `(cout << r)` returns `void`, and `void << " + "` is meaningless.

The second `<<` needs a stream as its left operand.

Returning `std::ostream&` is structurally required for chaining, not optional style.

Type Conversion and explicit - Challenge

What does `1` silently become without `explicit`?

```
// Without explicit:
class Rational {
    int num_, denom_;
public:
    Rational(int n) noexcept : num_{n}, denom_{1} {}    // single-arg, NOT explicit
    Rational(int num, int denom) : num_{num}, denom_{denom} {
        if (denom == 0) throw std::out_of_range("...");
    }
    bool operator==(const Rational& rhs) const {
        return num_ * rhs.denom_ == denom_ * rhs.num_;
    }
};

Rational r(3, 4);
if (r == 1) { /* ... */ }    // What is actually compared?
```

Answer: `1` is silently converted to `Rational(1) = 1/1`, then compared with `3/4`.

This might be intentional or a quiet bug.

With `explicit`, this is a compile error, forcing the auditable form `r == Rational(1)`. Silent implicit conversions are one of C++'s most notorious sources of subtle bugs.

Friends - Challenge

Can a friend function bypass the constructor's invariant check?

```
#include <stdexcept>
class Rational {
    friend void tamper(Rational& r);
    int num_, denom_;
public:
    Rational(int num, int denom) {
        if (denom == 0)
            throw std::out_of_range("...");
    }
};

void tamper(Rational& r) { r.denom_ = 0; } // can a friend do this?

Rational r(2, 3); tamper(r); // what is r now?
```

Answer: Yes, `tamper` compiles and runs, directly setting `denom_` to 0.

The constructor's check is irrelevant => the friend skips it entirely. `r` is now 2/0, an invalid object.

This is why accessors are safer than friends: they can't accidentally break invariants.

Helper Functions - Challenge

What happens when two .cc files define the same free function name?

```
// rational.cc
int computeGCD(int a, int b) {
    while (b) { int t = b; b = a % b; a = t; }
    return a;
}

// date.cc => different file, same project
int computeGCD(int x, int y) { /* different impl */ }
```

Answer: Linker error => two definitions of `computeGCD` in the global namespace cause a “multiple definition” conflict.

Fix: wrap in an anonymous namespace (`namespace { ... }`) inside each `.cc` file, making each definition invisible outside its translation unit.

override and final - Example

```
class Animal {
public:
    virtual void makeSound() const { cout << "Generic sound\n"; }
    virtual ~Animal() = default; // virtual destructor => always
};

class Dog : public Animal {
public:
    void makeSound() const override { cout << "Woof!\n"; }
    // void makeSond() const override {} // compile error: typo caught!
};

class ElectricCar final : public Vehicle { /* ... */ };
// class Tesla : public ElectricCar { }; // compile error: final class!
```

override and final - Challenge

What does missing virtual on the destructor actually cost you?

```
class Animal {
public:
    void makeSound() const { cout << "Generic sound\n"; } // NOT virtual
    ~Animal() { cout << "Animal destroyed\n"; } // NOT virtual
};

class Dog : public Animal {
public:
    void makeSound() const { cout << "Woof!\n"; }
    ~Dog() { cout << "Dog destroyed\n"; }
};

Animal* a = new Dog();
a->makeSound(); // What prints?
delete a; // What prints? Is anything leaked?
```

Answer: `a->makeSound()` prints “Generic sound” => the Dog version is silently ignored because `makeSound` is not virtual.

`delete a` calls only `~Animal()` => `~Dog()` never runs. If `Dog` allocated heap memory, it is now leaked with no warning.

Fix: mark both `makeSound` and `~Animal` as `virtual`. A non-virtual destructor in a base class is always a latent bug.

UNIVERSITY OF
WATERLOO



FACULTY OF MATHEMATICS
QUESTIONS?

THANK YOU!

tmellopaiva@uwaterloo.ca